Scholé Growth Lab — Study Guide

Everything you built, why, and what's expected — explained layer by layer.

> ***How to read this.*** *Each concept appears three times, getting deeper each time:* ***① Plain English*** *(the idea),* ***② The mechanism*** *(how it actually works),* ***③ Our numbers*** *(the exact values in this build). If you only read the ① lines top to bottom, you'll understand the whole thing. Read ② and ③ when you want to defend it.*

Live app: https://schole-growth-lab.vercel.app **Code:** `~/Desktop/schole-growth-lab/` · **Run the model:** `npx tsx scripts/calibrate.ts`

Table of contents

---

0. The 30-second picture

You built a web app that answers a marketing question — *"which version of a landing page makes the most people click the button, and can a system figure that out and design a better page by itself?"* — but it answers it with **simulated visitors** instead of real ones.

The app:

1. Shows **5 different landing pages** for the same product (Scholé).
2. Sends thousands of **fake-but-realistic visitors** through them.
3. **Learns** which messages, buttons, and layouts convert — with statistical confidence.
4. **Designs a brand-new page** that should beat all five, and **proves** it does.
5. Repeats the loop, and shows when further improvement **stops paying off**.

The whole thing is honest about being a simulation, and honest about *how sure* it is. That honesty is the point: the company is grading **how you think about experimentation**, not how flashy the demo is.

---

1. The task (what they asked for)

Who is Scholé? A San-Francisco AI startup that teaches companies' employees how to use AI, with short adaptive lessons and dashboards. Founded by two PhDs (Vinitra "Vini" Swamy, Paola Mejia). They raised

$3M. The role is **Founding Growth Engineer** — the person who builds systems to grow the company (experiments, funnels, landing pages, automation).

The exact brief. Build and host a web app exploring *how landing pages can auto-improve over time through experimentation and simulated user behavior.* Specifically it must let them see:

#	Required to show	Where our app shows it
1	The initial landing page versions	"The five contenders" gallery
2	How the pages were compared	"How the pages were compared" (bandit + charts)
3	The simulated user behavior	"The simulated user behavior" (heatmap + segments)
4	Which versions performed better	The leaderboard
5	The new generated variation(s)	"The generated variation" + targeted pages
6	A short explanation of what changed and why	"What changed, and why" (cites coefficients)

What they're really grading (from the brief, paraphrased): *"how you think about experimentation and how a system can learn from user behavior over time"* — and *"the best candidates fulfill the requirements and then go beyond."*

The deliverables you owe them:

- A **3-minute explainer video** (be honest — how it works + limitations).
- A **3-minute pitch video** (be convincing — why it's valuable).
- The **link to the hosted app**.
- Sent to **team@schole.ai**, subject `[Schole Challenge] Founding Growth Engineer <Your Name>`.
- **Heads-up:** a later technical interview will ask you to **modify the code live, under time pressure** — so you need to know the code (Part 8).

---

2. The big idea, in plain English

The problem. You have one product and five ways to sell it. Which page wins? Normally you'd guess, or run an A/B test for weeks with real traffic. We don't have real traffic, so we **simulate** it — and we make the simulation smart enough that a system can *learn* from it.

Our loop — five steps that repeat:

> ***Explore → Fit → Optimize → Validate → Iterate***

1. **Explore** — try lots of page "recipes" and watch how simulated people react.
2. **Fit** — build a little statistical **model** of what makes people convert (which message, which button, how much proof, how long the page).
3. **Optimize** — use that model to **design the best possible new page**.
4. **Validate** — put the new page back in the race and confirm it actually wins.
5. **Iterate** — repeat until a new page stops beating the champion (improvement plateaus).

The honesty twist — a "known-answer testbed." We *designed* the fake visitors, so in a sense we already know the answer. That's not cheating if you're upfront about it: we **planted a ground truth** and then

checked that the learning machinery **rediscovers it from behavior alone**. Like testing a metal detector by burying a coin yourself — you're not proving coins exist, you're proving the detector works. In production you'd swap the simulator for real traffic; the machinery is identical.

An analogy. It's a **flight simulator for landing pages**. Pilots train in a simulator before a real plane because it's cheaper, faster, and you can crash safely. Same here: the growth system learns to "fly" (pick and design winning pages) in a simulator before you point it at real, expensive traffic.

---

3. The concepts, layer by layer

3a. The pages and their "angles"

① **Plain English.** A landing page sells the same product five different ways. Each page leads with a different *angle* — the main argument it makes.

② **The mechanism.** Each page is data: a headline, a stack of sections, a call-to-action (CTA), and a set of numeric **features** the system can read (how much it emphasizes each angle, which CTA, how much social proof, how specific/number-heavy, how long).

③ **Our numbers.** Five angles and five pages (`lib/pages.ts`):

Page	Angle	Lead message	CTA
A	**ROI & Proof**	"Measure AI adoption. Prove the ROI."	Book a demo
B	**Adoption-Gap Pain**	"Your team has ChatGPT. They still don't use it."	Book a demo
C	**Role Personalization**	"One AI course for everyone teaches no one."	Book a demo
D	**Research Credibility**	"Built on a decade of learning science."	Learn more (soft)
E	**Speed / Micro-learning**	"Real AI skills in two-minute lessons."	Try it now (trial)

3b. The hidden buyer model (the personas / "DGP")

① **Plain English.** Behind the fake visitors are **three types of buyer**, each wanting different things. An exec wants proof and ROI. An HR/training lead wants personalization. A hands-on employee wants speed. The system **never sees these types directly** — it only sees how they behave.

② **The mechanism.** "DGP" = **Data-Generating Process**, the hidden rules that turn a visitor into a click. Each persona has *preferences* (how much they like each angle, which CTA, tolerance for long pages, appetite for proof/numbers). For a given (persona, page) we compute a **utility score** — basically `preferences · page features` — then squash it through a **logistic curve** (the S-shaped function that turns any score into a probability between 0 and 1) and add a little random noise. That probability is their chance of converting. The optimizer is **forbidden from seeing this** — that separation is what makes the learning real and not circular.

③ **Our numbers** (`lib/personas.ts` , `lib/constants.ts`). Three personas, traffic mix **30% / 40% / 30%**:

Persona	Share	Loves	Dislikes	Prefers CTA
Skeptical Exec	30%	ROI (+0.95), Research (+0.7), numbers	Speed (−0.35), long pages	Book a demo
L&D / People Lead	40%	Personalization (+0.95), Pain (+0.8), social proof	—	Book a demo
Hands-on IC	30%	Speed (+0.95), Personalization (+0.7)	Research (−0.3), ROI (−0.2)	Try it now

The utility formula lives in `trueLogit()` (`lib/simulate.ts`): angle affinity (weight `W_ANGLE = 4.2`), a CTA match bonus/penalty, proof and specificity bonuses, a length penalty, plus per-visit noise (`NOISE_SD = 0.45`). Because the **40% L&D group loves personalization** and the others don't hate it, personalization is the **consensus winner** — which is exactly what the system later discovers.

3c. The behavior signals

① **Plain English.** A simulated visitor doesn't just "convert or not." They scroll, linger on sections they like, skip ones they don't, sometimes bounce immediately, and maybe click the button.

② **The mechanism** (`simulateVisit()`). The visitor goes section by section. Interesting sections get more **dwell time**; boring ones get skipped and can trigger an early exit. How far they got = **scroll depth**. Whether they click is the conversion probability **gated by engagement** — you can't click a button you never scrolled to (there's a floor so the top CTA still counts).

③ **Our numbers.** Signals recorded per visit (`lib/types.ts → Visit`): per-section dwell seconds, scroll depth (0–1), time on page, bounced (yes/no), converted (yes/no), and the persona tag (treated as an observable "traffic segment," like a UTM source). Base dwell `BASE_DWELL = 6s` ; scroll gate floor `0.35` .

3d. Comparing the pages — the multi-armed bandit (Thompson sampling)

① **Plain English.** Imagine five slot machines ("one-armed bandits") with unknown payouts. You want to win the most while *figuring out* which is best. A **multi-armed bandit** does this by sending more and more visitors to whatever is looking best, while still occasionally testing the others. It's smarter than a fixed 20/20/20/20/20 split because it stops wasting traffic on clear losers.

② **The mechanism — Thompson sampling.** For each page we keep a **belief** about its true conversion rate, as a **Beta distribution** (a bell-ish curve over "what the rate might be," updated by every win/loss). For each visitor: draw one random guess from each page's belief, **serve the page with the highest guess**, then update that page's belief with what happened. Good pages get sampled high more often → they get more traffic → their belief sharpens. This is `runExperiment()` (`lib/bandit.ts`).

③ **Our numbers.** 14 rounds × 300 visitors = **4,200 visitors**. The bandit concentrates almost all traffic on page **C**. Final realized conversion (the bandit's own blended rate): **14.81%**.

3e. Honest baselines — floor, ceiling, and regret

① **Plain English.** "14.81%" means nothing alone. Compared to what? We show two honest reference lines: the **floor** (what a dumb even split would get) and the **ceiling** (what a perfect oracle who always served the single best page would get). The bandit should sit between them, climbing toward the ceiling.

② **The mechanism.** *Even-split* baseline = the **average** of the pages' true rates. *Oracle/best-arm* = the **highest** true rate. **Regret** = how many conversions you *lost* versus the oracle while you were still learning. Crucially we compute these from a **clean, equal-traffic measurement pass** (`evaluateField()` in `lib/lab.ts`), *not* from the bandit's own lopsided data — otherwise the comparison would be rigged in the bandit's favor (this was a real bug we fixed; see Part 7).

③ **Our numbers.** Even split **12.03%** (floor) · bandit **14.81%** · oracle **17.00%** (ceiling). Cumulative **regret ≈ 63 conversions** vs the oracle. The honest takeaway: *"the bandit finds the winner fast and closes most of the gap to the oracle,"* not the strawman *"+23% over uniform."*

3f. Are we sure? — significance

① **Plain English.** Even after the test, maybe the "winner" is just lucky. We don't declare a winner until we're statistically confident.

② **The mechanism.** Two tools. **Wilson confidence intervals** (`wilson()` in `lib/rng.ts`) put an honest error band around each page's rate. **Posterior P(best)** runs a quick Monte-Carlo over the Beta beliefs: sample all pages thousands of times, count how often each comes out on top. We only stamp "WINNER ✓ sig" when the leader's P(best) clears **95%**; otherwise it says *"leading · not yet sig."*

③ **Our numbers.** Page C: **P(best) = 98%** → significant. Everyone else ≤ 1%.

3g. Learning *why* — the response model (the heart of v2)

① **Plain English.** Picking the winning page isn't the same as understanding *why* it won. We fit a small **model of conversion** that scores each ingredient: which angle helps, which button helps, does proof help, do longer pages hurt — each with a **confidence interval** (how sure we are).

② **The mechanism — logistic regression on a factorial design.** Two ideas:

- **Logistic regression**: fit an equation `P(convert) = logistic(b0 + b1·feature1 + b2·feature2 + …)` . Each `b` (**coefficient**) is the effect of that feature on conversion, in log-odds. We fit it with **IRLS** (Iteratively Reweighted Least Squares — Newton's method for logistic models) plus a touch of **ridge** regularization for stability (`fitLogistic()` in `lib/regression.ts` , with mini matrix math in `lib/linalg.ts`). Standard errors → 95% CIs.
- **Factorial design** (`lib/design.ts`): here's the subtle part. On the 5 real pages, the CTA is *tangled up* with the angle (e.g. only the worst page uses the soft CTA), so you can't tell whether "soft CTA is bad" or "that page was bad." So we generate a separate **exploration grid** of **60 synthetic pages** where angle, CTA, proof, specificity, and length each vary **independently**. That **de-confounds** them — now the model can isolate each effect cleanly. This is the single most important idea in the whole build.

③ **Our numbers** (fitted on **36,000** exploration visits — `npx tsx scripts/calibrate.ts`):

Feature	Coefficient	Reading
Personalization angle	**+1.03**	strongest converting angle
Adoption-gap pain	+0.51	helps
ROI	+0.06	≈ neutral
Research	−0.79	hurts
Speed	−0.81	hurts most (weakest)

CTA "Book a demo"	**+0.65**	best CTA (vs soft baseline), *measured independent of angle*
CTA "Try it now"	+0.14	slightly better than soft
Social proof	**+0.47**	proof lifts conversion
Specificity (numbers)	+0.23	concrete numbers help
Page length	**−0.49**	longer pages convert worse

3h. Generating the new page — argmax of the model

① **Plain English.** Now we have a scoring model, we just ask it: *"of all the page recipes you could build, which scores highest?"* — and build that one.

② **The mechanism.** `optimizeVariant()` (`lib/variant.ts`) enumerates the realistic combinations (lead angle × support angle × CTA) and picks the **argmax** of the fitted model. The scalar levers (proof, specificity, length) are set to the **end of the explored range that the coefficient's sign prefers** — proof coefficient is positive → max proof; length coefficient is negative → short page. **Nothing is hand-picked.** An LLM then writes the words into this fixed, data-chosen structure (3k). The "what changed & why" notes literally quote the coefficients + CIs.

③ **Our numbers.** The optimizer produces **V1**: lead **Personalization**, support **Pain**, CTA **Book a demo**, **max** proof + specificity, **short** (5 sections). Model **predicts 21.3%**; **observed 19.9%** vs the best original (C) at **17.7%** in the same measurement pass → **+12.4%**.

3i. Multi-round evolution — and when it stops

① **Plain English.** "Auto-improve over *time*" means more than one step. We let it generate a new page each round, keep it only if it beats the current champion, and watch the gains shrink to zero.

② **The mechanism.** `runEvolution()` tries the next-best hypothesis each round and **accepts only if it beats the incumbent** by a margin. Round 1's big winner gets kept; later rounds try weaker angles and get **rejected** → the "frontier" rises then **plateaus**. This proves there's real *selection pressure*, not a guaranteed win.

③ **Our numbers.** R1 Personalization **19.9% → ACCEPT** (frontier jumps from 17.7%). R2 Pain reject · R3 ROI reject · R4 Research reject → frontier holds at 19.9%.

3j. Per-segment targeting (the "go beyond")

① **Plain English.** Different buyers want different things — so instead of one page for everyone, build a page tuned to each segment.

② **The mechanism.** We fit a **separate model per segment** and optimize a page against each one (`optimizeVariant(..., {forSegment})`). Then we measure each segment's conversion on the generic winner vs the three targeted pages.

③ **Our numbers.** Exec → an **ROI** page (and converts ≈42% on it vs 6% on the generic). IC → a **Speed** page (≈31% vs 7%). The **L&D** segment is *already* best served by the generic winner (it's personalization-led, which they love) — so the system would **only ship the targeted pages that actually beat the generic one.** Honest, and a sharper insight than "targeting always wins."

3k. The LLM's (deliberately small) role

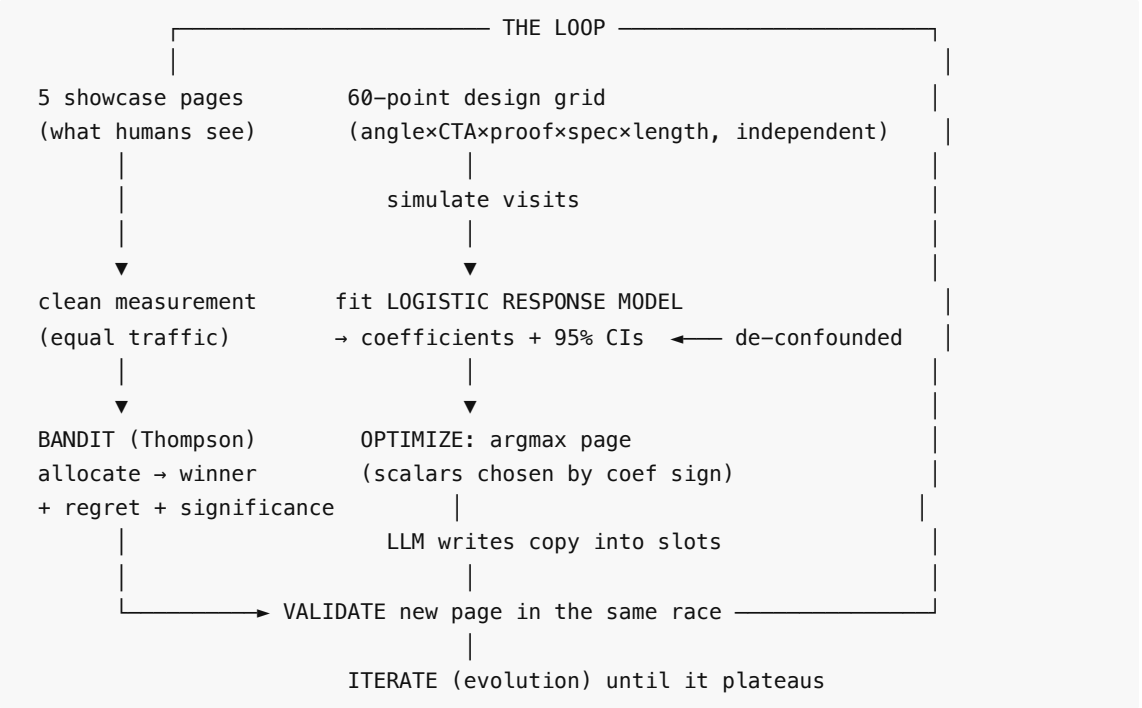
① **Plain English.** The AI writes the *words*, not the *strategy*.

② **The mechanism.** The strategy (angle, CTA, levers, structure) is 100% decided by the model/optimizer. The LLM (via **OpenRouter**, `app/api/generate-variant/route.ts`) only fills copy into the fixed slots. If there's no API key or the call fails, baked-in copy is used and the page is labelled "cached copy." This keeps the measurable features — the only thing that affects the simulated result — **always the data-derived ones.**

③ **Our numbers.** Conversion is identical whether the words come from the LLM or the fallback, because the words don't touch the feature vector. That's the honesty guarantee.

---

4. How it all fits together



The **hidden persona model** sits underneath everything generating behavior; the **optimizer only sees clicks/scroll/dwell**. Section 4 of the app ("What the system learned") is the model's coefficients; section 2 ("How compared") is the bandit; section 5 is the generated page.

---

5. The story the demo tells (screen by screen)

When you (or a reviewer) open the app, it **auto-runs** so nothing is blank. Top to bottom:

1. **Hero + loop diagram** — one-line promise ("landing pages that improve themselves") and the Explore→Fit→Optimize→Validate→Iterate strip. An **"All data simulated"** chip is right at the top.
2. **The five contenders** — five clickable cards; click any to open the *real, fully rendered* page. *(Proves requirement #1.)*
3. **How the pages were compared** — the allocation chart (traffic concentrating on C), the conversion chart with **floor / bandit / oracle** lines + **regret**, and the **leaderboard** with confidence intervals, **P(best)**, and a **significance** stamp. *(Requirements #2 + #4.)*
4. **The simulated user behavior** — the **dwell heatmap** and the **segment × page matrix** showing different segments prefer different pages. *(Requirement #3.)*

5. **What the system learned** — the **coefficient plot** (effect of each angle + lever, with CIs). This is the honest "why." *(Sets up #6.)*
6. **The generated variation** — the new page rendered live, the **"what changed & why"** notes quoting coefficients, an **"Is the win real?"** card (predicted vs observed vs **multi-seed lift +18%, 95% CI, 93% win-rate**), and a **re-run leaderboard** with the variant winning. *(Requirements #5 + #6.)*
7. **Iterate** — the **evolution** panel: 1 accepted, 3 rejected, frontier plateaus.
8. **Go further** — the **per-segment targeted pages** + matrix.
9. **Method & honest caveats** — the known-answer-testbed framing and what's illustrative.

---

6. The honesty backbone

These are the four things that make this credible (and your strongest talking points):

1. **It's a known-answer testbed.** We planted a ground truth (the personas) and proved the pipeline *recovers* it from behavior alone. The optimizer never sees the personas.
2. **The new page is earned, not constructed.** Its features are the **argmax of a fitted model**, with confidence intervals — and across 40 seeds it wins **≈93%** of the time, **not 100%**. Sometimes it loses, and we show that.
3. **Honest baselines.** Bandit framed by an even-split floor and an oracle ceiling, with **regret** and **significance gating** — never a bare max or a strawman "lift over uniform."
4. **Everything is seeded.** The same run reproduces exactly, so the demo is stable and you can reason about exact outputs live. Copy and customer logos are labelled **illustrative**.

---

7. From v1 (rigged) to v2 (rigorous) — why the architecture is what it is

You first built a simpler version, then had **four independent AI reviewers** (with no context) tear it apart. They converged on one verdict: *the system asserted its win by construction instead of demonstrating it.* Understanding this is the fastest way to understand *why* the code looks the way it does now.

v1 problem the reviewers found	v2 fix (what you now have)
Variant's proof/specificity were **hard-coded to max** (the exact levers that help) → rigged win	Features are the **argmax of a fitted model**; levers chosen by coefficient sign, with CIs
"Best CTA" was **confounded** (each CTA lived on a different page)	**60-point factorial design** varies CTA independently of angle → de-confounded
Lift compared **different seeds + different fields** (+27% vs real +15%)	**Same-pass** measurement; honest lift
"Lift over even split" is a **tautology**	Report **regret** + oracle ceiling + best-arm
Winner declared by **bare max**, no significance	**P(best) ≥ 95%** gate + Wilson intervals
One generation step called "improvement over time"	**Multi-round evolution** that plateaus and **rejects**
One seed → fragile number	**Multi-seed** distribution (mean + CI + win-rate)
Per-page results shifted when you added a page (shared RNG)	**Per-page RNG isolation** (seeded by page id)

API could 500; fake LLM copy mislabeled	Route hardened; only labels "by {model}" on full output
Fabricated customer logos as fact	Relabeled **illustrative**; "All data simulated" chip

The lesson you can say out loud: *"My first version was a good demo but it was riggable — so I rebuilt the learning core around a fitted model and honest statistics. The same codebase went from 'competent toy' to 'demonstrably honest experimentation.'"*

---

8. Code map (for the live-edit interview)

TypeScript + Next.js (App Router). The **logic is all in `lib/`** (pure, framework-free, easy to edit); the UI just renders it. Know these cold:

The hidden world (the simulator):

- `lib/personas.ts` — the 3 buyer types + their preferences. *Tweak a preference here and the whole story can shift.*
- `lib/constants.ts` — the DGP weights (`W_ANGLE` , `NOISE_SD` , scroll/dwell knobs). The "physics."
- `lib/simulate.ts` — `trueLogit()` (hidden utility), `simulateVisit()` (one visit → signals), `sampleBehavior()` (equal-traffic measurement; **per-page seeded RNG**).
- `lib/rng.ts` — seeded PRNG, Beta/Gamma sampling, `wilson()` intervals, `hashStringToSeed()` .

The learning core:

- `lib/design.ts` — `buildExplorationDesign()` → the 60-point factorial grid; `blendWeights()` .
- `lib/regression.ts` — `featurize()` , `fitLogistic()` (IRLS + ridge → coefficients, SEs, CIs).
- `lib/linalg.ts` — tiny matrix inverse/multiply for the regression.
- `lib/learn.ts` — `fitResponseModel()` (population + per-segment fits).
- `lib/insights.ts` — `summarizeFit()` , `computeInsights()` (turns the fit into rankings + segments).
- `lib/variant.ts` — `optimizeVariant()` (argmax page), rationale, copy realization.
- `lib/bandit.ts` — `runExperiment()` (Thompson + regret + P(best) significance).

Orchestration + UI:

- `lib/lab.ts` — **the conductor.** `runLab()` , `evaluateField()` , `evaluateVariant()` (same-pass lift), `runEvolution()` , `multiSeedLift()` , and `LAB_CONFIG` (every seed and size in one place).
- `app/page.tsx` — the dashboard; calls the `lib/lab.ts` functions and renders sections.
- `components/` — `charts.tsx` (recharts), `panels.tsx` (leaderboard, coefficient plot, heatmap, segment matrix, evolution), `variant-reveal.tsx` , `gallery.tsx` , `LandingPagePreview.tsx` .
- `app/api/generate-variant/route.ts` — the OpenRouter call + fallback.
- `scripts/calibrate.ts` — **run this** to see the whole model's behavior in one shot.

The "magic constants" most likely to come up in a live edit (`lib/lab.ts → LAB_CONFIG`): `rounds: 14` , `visitorsPerRound: 300` , `explorePerPage: 600` , `showcasePerPage: 1500` , `multiSeedK: 40` , and the seeds (`banditSeed: 42` , etc.). Changing a persona preference in `lib/personas.ts` or a `DGP` weight in `lib/constants.ts` is the highest-leverage edit — it changes which page wins. After any

logic change, run `npx tsx scripts/calibrate.ts` to confirm the story still holds, then `npm run build`.

---

9. The two videos (talking points + outline)

Both ≈3 minutes. Record the **app on screen**; talk over it. Don't read these verbatim — they're beats.

Explainer video (be honest)

Goal: show you understand experimentation and were honest about a simulation.

Time	On screen	Say (beat)
0:00–0:20	Hero + loop diagram	"Same product, five landing pages. The system learns what converts and designs a better one — on simulated traffic, and I'll be honest about exactly how."
0:20–0:50	The 5 contenders; open one	"Five distinct angles — ROI, pain, personalization, research, speed. Real rendered pages, different CTAs."
0:50–1:20	Behavior + segment matrix	"Behind it is a hidden model of three buyer personas. The optimizer never sees them — only clicks, scroll, dwell. Notice different segments prefer different pages."
1:20–1:55	Coefficient plot	"It fits a logistic model on a 60-point design that varies CTA, proof, and length *independently of angle* — so these effects are de-confounded, with confidence intervals. Personalization and the demo CTA win; longer pages hurt."
1:55–2:25	Generated page + 'what changed'	"The new page is the argmax of that model — every choice traces to a coefficient. The LLM only writes the copy into a data-chosen structure."
2:25–2:50	'Is the win real?' + evolution	"Honest accounting: it beats the best original, but across 40 seeds it wins 93% of the time, not 100. Evolution shows it rising then plateauing — real selection pressure."
2:50–3:00	Caveats footer	"It's a known-answer testbed — I planted a ground truth and proved the pipeline recovers it. Swap in real traffic and the loop is identical."

Pitch video (be convincing)

Goal: make them want this person on the growth team.

Time	On screen	Say (beat)
0:00–0:25	Hero	"Every growth team guesses at landing pages and waits weeks on A/B tests. What if the page improved itself — and told you *why*?"
0:25–1:00	Race + leaderboard climbing	"This is that system. It runs the experiment, finds the winner fast with low regret, and knows when it's statistically sure."

1:00–1:45	Coefficient plot → generated page	"It doesn't just pick a winner — it learns the *recipe* for conversion and designs a brand-new page from it. Here it beats the best human-written page by double digits."
1:45–2:20	Segment targeting	"And it goes further: it discovers different buyers want different stories and writes a page for each — which is literally Scholé's own personalization thesis, applied to growth."
2:20–2:50	Evolution + honesty	"It improves round over round until it plateaus, and it's honest about uncertainty — exactly what you want running real budget."
2:50–3:00	Live URL	"It's live, it's reproducible, and the same engine points straight at real traffic. I'd love to build this for Scholé."

---

10. Submission checklist

- [ ] Record + upload **explainer** (3:00) and **pitch** (3:00) — unlisted YouTube / Loom links.
- [ ] (Optional) add `OPENROUTER_API_KEY` in Vercel → Settings → Environment Variables, redeploy, so the hosted app does **live** LLM copy (you enter the key; the app works without it too).
- [ ] Final smoke test of https://schole-growth-lab.vercel.app (run, generate, evolve, targeted).
- [ ] Email **team@schole.ai**, subject `[Schole Challenge] Founding Growth Engineer <Your Name>`, with: the two video links + the app link + one or two lines of intro.
- [ ] Due **Saturday, July 4**.

Appendix A — Glossary

- **Angle** — the main argument a page leads with (ROI, pain, personalization, research, speed).
- **Argmax** — "the input that gives the maximum output"; here, the best-scoring page recipe.
- **Bandit (multi-armed)** — an algorithm that splits traffic to maximize wins while learning which option is best.
- **Beta distribution** — a probability curve over "what a conversion rate might be"; the bandit's belief per page.
- **Coefficient** — a number in the fitted model = the effect of one feature on conversion (in log-odds).
- **Confidence interval (CI)** — the honest range a number probably lies in (we use 95%).
- **Conversion** — the goal action; here, clicking the call-to-action.
- **CTA** — Call To Action (the button: "Book a demo," "Try it now," "Learn more").
- **De-confounding** — separating tangled effects so you can credit each one correctly.
- **DGP** — Data-Generating Process: the hidden rules that turn a visitor into a click.
- **Dwell time** — how long a simulated visitor spends on a section.
- **Factorial design** — a test grid where each factor varies independently of the others.
- **IRLS** — Iteratively Reweighted Least Squares; how we fit the logistic regression.
- **Logistic regression** — a model that predicts a probability from weighted features via an S-curve.
- **Oracle / best arm** — a hypothetical that always serves the single best page (the ceiling).
- **P(best)** — the posterior probability that a given page is truly the best; our significance gate.
- **Regret** — conversions lost vs the oracle while still learning.
- **Ridge** — a small penalty that keeps regression coefficients stable.
- **Seed** — the number that makes the random simulation reproducible.
- **Segment** — an observable group of visitors (here, the persona tag, like a UTM source).
- **Thompson sampling** — the bandit rule: sample each belief, serve the highest, update.

- **Wilson interval** — a reliable confidence interval for a proportion (a conversion rate).

Appendix B — Architecture diagram

See the box diagram in **Part 4**. In one sentence: *a hidden persona model generates behavior → an even-traffic pass + a 60-point design feed a logistic response model → the model's argmax becomes a new page → a bandit validates it with regret + significance → evolution iterates until it plateaus.*

Appendix C — Key-numbers cheat sheet

Thing	Value
Pages / personas / traffic mix	5 / 3 / 30%-40%-30%
Exploration design	60 points (5 angles × 3 CTAs × 4 scalar combos), 36,000 visits
Showcase leaderboard	C 17.0% · B 13.0% · E 12.8% · A 12.7% · D 4.7%
Top fitted coefficients	personalization +1.03 · demo CTA +0.65 · social proof +0.47 · length −0.49
Weakest angle (dropped)	speed (−0.81)
Bandit vs floor vs ceiling	14.81% vs 12.03% vs 17.00%; regret ≈ 63
Winner significance	page C, P(best) = 98%
Generated variant (V1)	personalization + pain + demo, max proof/spec, short
V1 predicted / observed / lift	21.3% / 19.9% vs 17.7% = **+12.4%** (same pass)
Multi-seed lift	mean **+18%**, 95% CI [−4%, +51%], **win-rate 93%** (n=40)
Evolution	R1 accept (19.9%), R2–R4 reject → plateau
Segment winners	Exec→ROI · L&D→Personalization · IC→Speed

Built for the Scholé Founding Growth Engineer challenge. All data simulated; numbers illustrative.